Embedded Linux application development

Contents

Developing applications on embedded Linux

Application development

- An embedded Linux system is just a normal Linux system, with usually a smaller selection of components
- In terms of application development, developing on embedded Linux is exactly the same as developing on a desktop Linux system
- All existing skills can be re-used, without any particular adaptation
- All existing libraries, either third-party or in-house, can be integrated into the embedded Linux system
  - Taking into account, of course, the limitation of the embedded systems in terms of performance, storage and memory
- Application development could start on x86, even before the hardware is available.

Leverage existing libraries and languages

- Many developers getting started with embedded Linux limit themselves to C, sometimes C++, and the C/C++ standard library.
- However, there are a lot of libraries and languages that can help you accelerate and simplify your application development
  - Compiled languages like Rust and Go are increasingly popular
  - Interpreted languages, especially Python
  - Higher-level libraries: Qt, Glib, Boost, and many more
- Make sure to evaluate what is the right choice for your project, but pay attention to
  - Footprint and performance on low-end platforms
  - Use well-maintained and well-known technologies

Building your applications/libraries

- Even for simple applications or libraries, make use of a build system
  - `CMake`
  - `Meson`
- This will simplify
  - the build process of your application
  - the life of developers joining your project
  - the packaging of your application into an embedded Linux build system

Getting started with *meson*

Minimal `meson.build`

```
project('example', 'c')
executable('demo', 'main.c')
```

`meson.build` for multiple programs and source files

```
project('example', 'c')
src_demo1 = ['demo1.c', 'foo1.c']
executable('demo1', src_demo1)
src_demo2 = ['demo2.c', 'foo2.c']
executable('demo2', src_demo2)
```

Options with *meson*

meson_options.txt

```
option('demo-debug', type : 'feature', value : 'disabled')
```

meson.build

```
project('tutorial', 'c')
demo_c_args = []
if get_option('demo-debug').enabled()
   demo_c_args += '-DDEBUG'
endif executable('demo', 'main.c', c_args: demo_c_args)
```

Library dependencies with *meson*

`meson.build`

```
project('tutorial', 'c')
gtkdep = dependency('gtk+-3.0')
executable('demo', 'main.c', dependencies : gtkdep)
```

The dependency `gtk+-3.0` is searched using `pkg-config`.

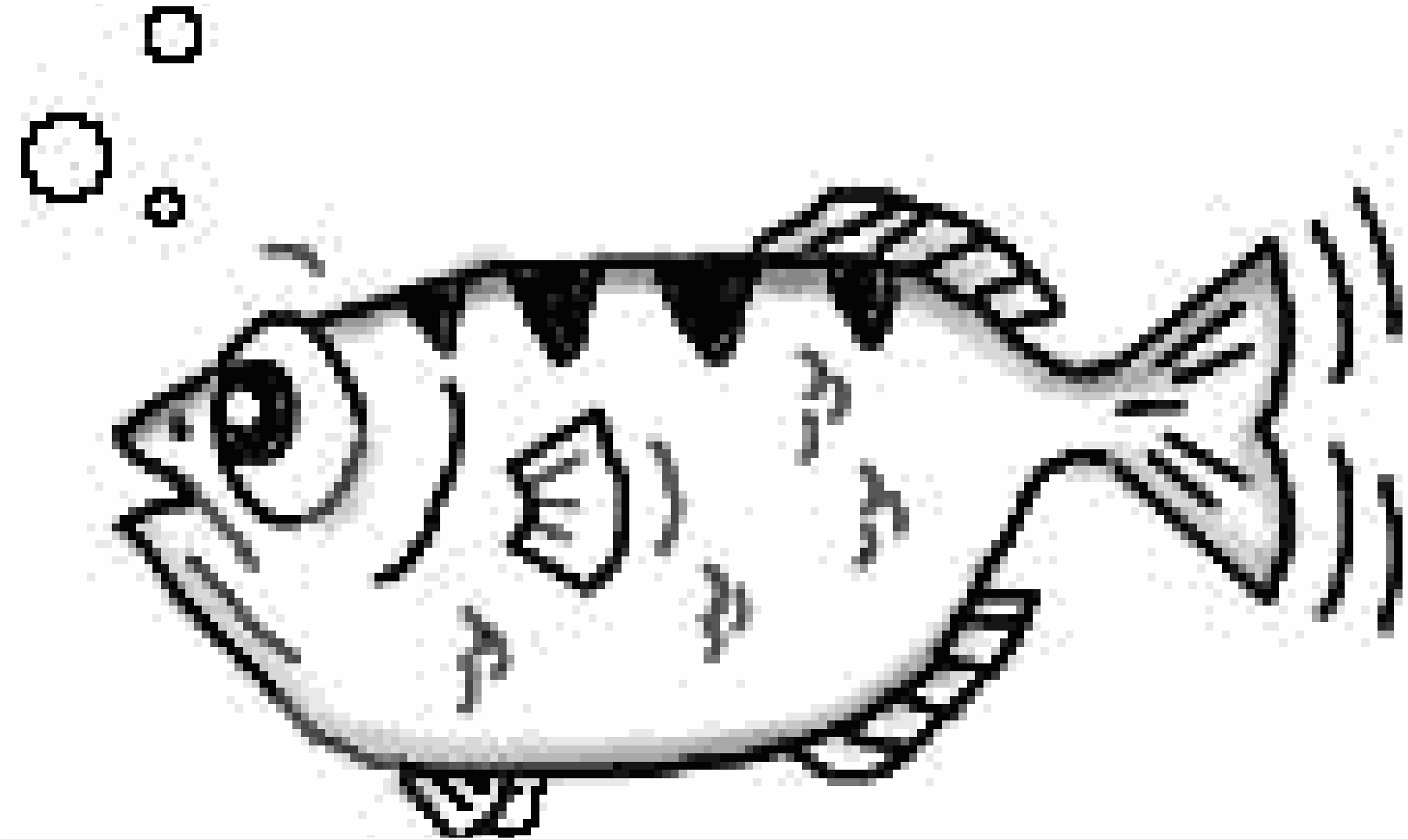
Debugging

GDB: GNU Project Debugger

GDB: GNU Project Debugger

- The debugger on GNU/Linux, available for most embedded architectures.
- Supported languages: C, C++, Pascal, Objective-C, Fortran, Ada...
- Command-line interface
- Integration in many graphical IDEs
- Can be used to
  - control the execution of a running program, set breakpoints or change internal variables
  - to see what a program was doing when it crashed: post mortem analysis
- https://www.gnu.org/software/gdb/
- https://en.wikipedia.org/wiki/Gdb
- New alternative: *lldb* (https://lldb.llvm.org/) from the LLVM project.

GDB: GNU Project Debugger



GDB crash course (1/3)

- GDB is used mainly to debug a process by starting it with *gdb*
  - `\$\_gdb <program>`
- GDB can also be attached to running processes using the program PID
  - `\$\_gdb -p <pid>`
- When using GDB to start a program, the program needs to be run with
  - `(gdb) run [prog_arg1 [prog_arg2] ...]`

GDB crash course (2/3) A few useful GDB commands

- `break foobar` (`b`) Put a breakpoint at the entry of function `foobar()`
- `break foobar.c:42` Put a breakpoint in `foobar.c`, line 42
- `print var`, `print \$reg` or `print task->files[0].fd` (`p`) Print the variable `var`, the register `\$reg` or a more complicated reference. GDB can also nicely display structures with all their members
- `info registers` Display architecture registers

GDB crash course (3/3)

- `continue` (`c`) Continue the execution after a breakpoint
- `next` (`n`) Continue to the next line, stepping over function calls
- `step` (`s`) Continue to the next line, entering into subfunctions
- `stepi` (`si`) Continue to the next instruction
- `finish` Execute up to function return
- `backtrace` (`bt`) Display the program stack

GDB advanced commands (1/3)

- `info threads` (`i threads`) Display the list of threads that are available
- `info breakpoints` (`i b`) Display the list of breakpoints/watchpoints
- `delete <n>` (`d <n>`) Delete breakpoint
- `thread <n>` (`t <n>`) Select thread number
- `frame <n>` (`f <n>`) Select a specific frame from the backtrace, the number being the one displayed when using `backtrace` at the beginning of each line

GDB advanced commands (2/3)

- `watch <variable>` or `watch <address>` Add a watchpoint on a specific variable/address.
- `break foobar.c:42 if condition` Break only if the specified condition is true
- `watch <variable> if condition` Trigger the watchpoint only if the specified condition is true
- `display <expr>` Automatically prints expression each time program stops
- `x/<n><u> <address>` Display memory at the provided address. `n` is the amount of memory to display, `u` is the type of data to be displayed (`b/h/w/g`). Instructions can be displayed using the `i` type.

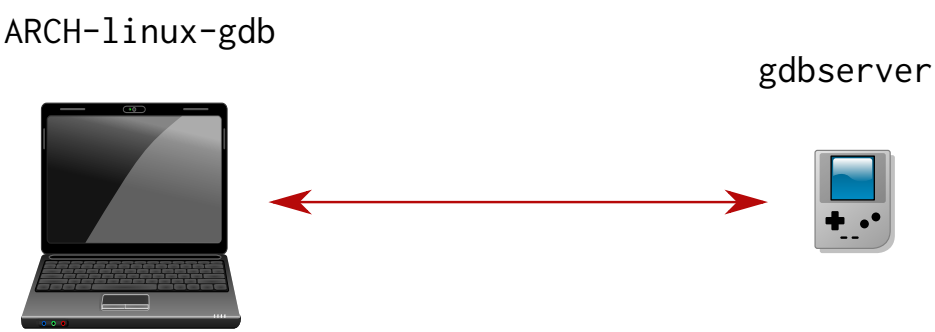
GDB advanced commands (3/3)

- `list <expr>` Display the source code associated to the current program counter location.
- `disassemble <location,start_offset,end_offset>` (`disas`) Display the assembly code that is currently executed.
- `print variable = value` (`p variable = value`) Modify the content of the specified variable with a new value
- `p function(arguments)` Execute a function using GDB. NOTE: be careful of any side effects that may happen when executing the function
- `p \$newvar = value` Declare a new gdb variable that can be used locally or in command sequence
- `define <command_name>` Define a new command sequence. GDB will prompt for the sequence of commands.

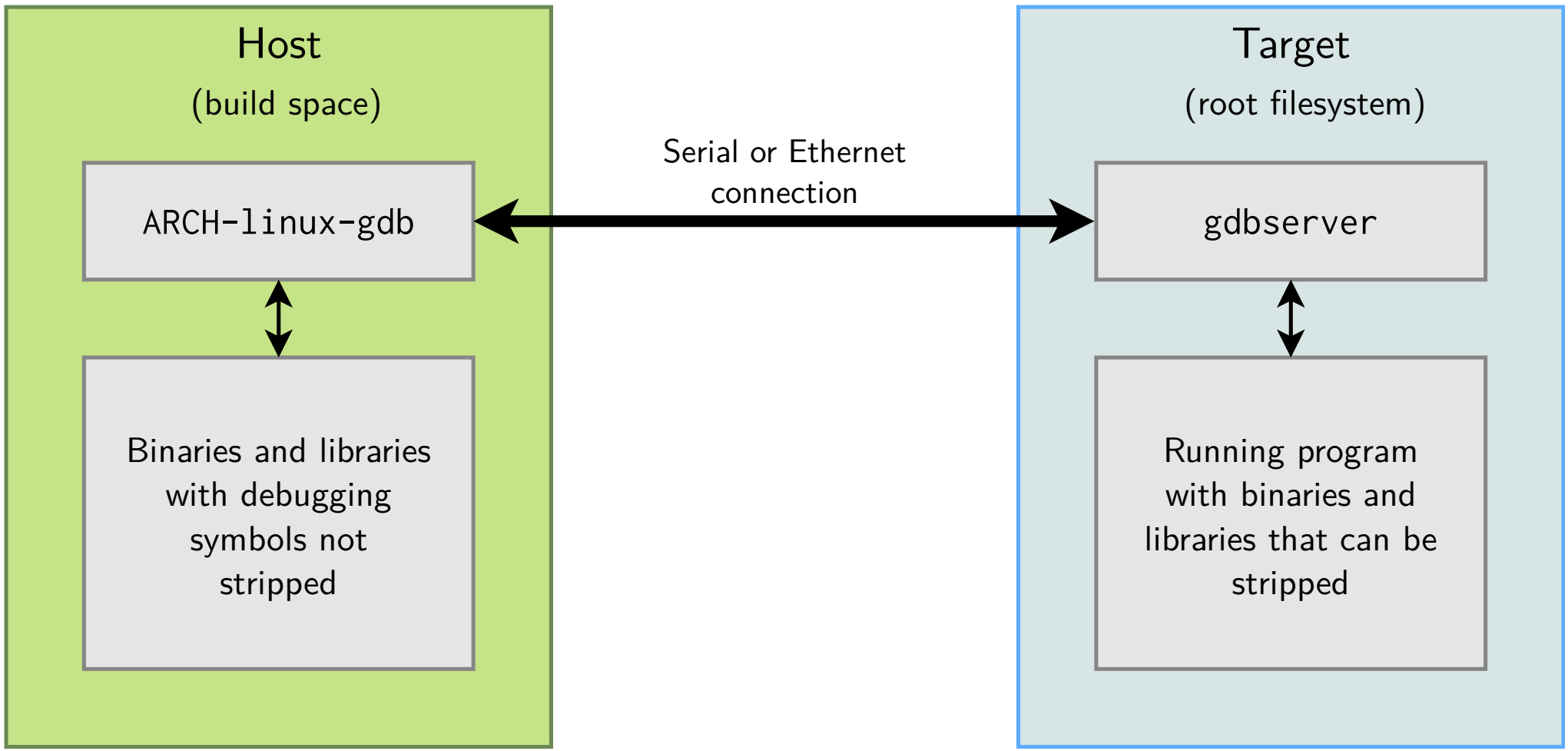
Remote debugging

Remote debugging

- In a non-embedded environment, debugging takes place using `gdb` or one of its front-ends.
- `gdb` has direct access to the binary and libraries compiled with debugging symbols, which is often false for embedded systems (binaries are stripped, without debug_info) to save storage space.
- For the same reason, embedding the `gdb` program on embedded targets is rarely desirable (2.4 MB on x86).
- Remote debugging is preferred
  - `ARCH-linux-gdb` is used on the development workstation, offering all its features.
  - `gdbserver` is used on the target system (only 400 KB on arm).



Remote debugging: architecture



Remote debugging: target setup

- On the target, run a program through `gdbserver`. Program execution will not start immediately. `gdbserver :<port> <executable> <args>` `gdbserver /dev/ttyS0 <executable> <args>`
- Otherwise, attach `gdbserver` to an already running program: `gdbserver –attach :<port> <pid>`
- You can also start gdbserver without passing any program to start or attach (and set the target program later, on client side): `gdbserver –multi :<port>`

Remote debugging: host setup

- Then, on the host, start `ARCH-linux-gdb <executable>`, and use the following `gdb` commands:
  - To tell `gdb` where shared libraries are: `gdb> set sysroot <library-path>` (typically path to build space without `lib/`)
  - To connect to the target: `gdb> target remote <ip-addr>:<port>` (networking) `gdb> target remote /dev/ttyUSB0` (serial link)
    - Make sure to replace `target remote` with `target extended-remote` if you have started gdbserver with the `–multi` option
  - If you did not set the program to debug on gdbserver commandline: `gdb> set remote exec-file <path_to_program_on_target>`

Coredumps for post mortem analysis

- It is sometime not possible to have a debugger attached when a crash occurs
- Fortunately, Linux can generate a `core` file (a snapshot of the whole process memory at the moment of the crash), in the ELF format. gdb can use this `core` file to let us analyze the state of the crashed application
- On the target
  - Use `ulimit -c unlimited` in the shell starting the application, to enable the generation of a `core` file when a crash occurs
  - The output name and path for the coredump file can be modified using `/proc/sys/kernel/core_pattern` (see man5core)
    - Example: `echo /tmp/mycore > /proc/sys/kernel/core_pattern`
  - Depending on the system configuration, the `core_pattern` file may be rewritten automatically by some software to handle core files or even disable core generation (eg: systemd)
- On the host
  - After the crash, transfer the `core` file from the target to the host, and run `ARCH-linux-gdb application-binary core-file`

minicoredumper

- Coredumps can be huge for complex applications
- minicoredumper is a userspace tool based on the standard core dump feature
  - Based on the possibility to redirect the core dump output to a user space program via a pipe
- Based on a JSON configuration file, it can:
  - save only the relevant sections (stack, heap, selected ELF sections)
  - compress the output file
  - save additional information from `/proc`
- https://github.com/diamon/minicoredumper
- "Efficient and Practical Capturing of Crash Data on Embedded Systems"
  - Presentation by minicoredumper author John Ogness
  - Video: https://www.youtube.com/watch?v=q2zmwrgLJGs
  - Slides: elinux.org/images/8/81/Eoss2023_ogness_minicoredumper.pdf

Tracing and profiling

strace

System call tracer - https://strace.io

- Available on all GNU/Linux systems
  Can be built by your cross-compiling toolchain generator or by your build system.
- Allows to see what any of your processes is doing: accessing files, allocating memory... Often sufficient to find simple bugs.
- Usage: `strace <command>` (starting a new process) `strace -f <command>` (follow child processes too) `strace -p <pid>` (tracing an existing process) `strace -c <command>` (time statistics per system call) `strace -e <expr> <command>` (use expression for advanced filtering)

See the strace manual for details

strace

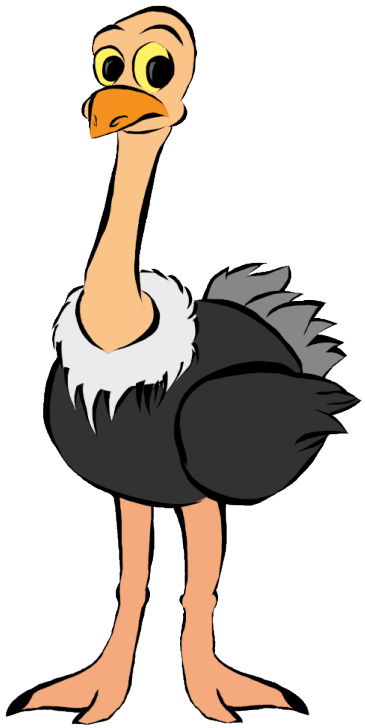


Image credits: https://strace.io/

strace example output

strace example output

Hint: follow the open file descriptors returned by `open()`. This tells you what files are handled by further system calls.

strace filtering

- Display only a specific set of system calls:

```
\$\_strace -e 'openat,write' cat Makefile
```

- Filter out specific system calls:

```
\$\_strace -e '!poll' cat Makefile
```

- Show only system calls returning a specific status

```
\$\_strace -e 'status=failed' cat Makefile
```

- Trace how a file is accessed and used among different system calls

```
\$\_strace -P '/etc/ld.so.cache' cat Makefile
```

- Run `strace –tips` to learn new commands !

ltrace A tool to trace **shared** library calls used by a

program and all the signals it receives

- Very useful complement to `strace`, which shows only system calls.
- Of course, works even if you don't have the sources
- Allows to filter library calls with regular expressions, or just by a list of function names.
- With the `-S` option it shows system calls too!
- Also offers a summary with its `-c` option.
- Manual page: `https://linux.die.net/man/1/ltrace`
- Works better with *glibc*. `ltrace` used to be broken with *uClibc* (now fixed), and is not supported with *Musl* (Buildroot 2022.11 status).

See `https://en.wikipedia.org/wiki/Ltrace` for details

ltrace example output

```
# ltrace  ffmpeg -f video4linux2 -video_size 544x288 -input_format mjpeg -i /dev
/video0 -pix_fmt rgb565le -f fbdev /dev/fb0
__libc_start_main([ "ffmpeg", "-f", "video4linux2", "-video_size"... ] <unfinished ...>
setvbuf(0xb6a0ec80, nil, 2, 0)                  = 0
av_log_set_flags(1, 0, 1, 0)                    = 1
strchr("f", ':')                                = nil strlen("f")                                = 1
strncmp("f", "L", 1)                            = 26
strncmp("f", "h", 1)                            = -2
strncmp("f", "?", 1)                            = 39
strncmp("f", "help", 1)                         = -2
strncmp("f", "-help", 1)                        = 57
strncmp("f", "version", 1)                      = -16
strncmp("f", "buildconf", 1)                    = 4
strncmp("f", "formats", 1)                      = 0
strlen("formats")                               = 7
strncmp("f", "muxers", 1)                       = -7
strncmp("f", "demuxers", 1)                     = 2
strncmp("f", "devices", 1)                      = 2
strncmp("f", "codecs", 1)                       = 3
...
```

ltrace summary Example summary at the end of the

ltrace output (`-c` option)

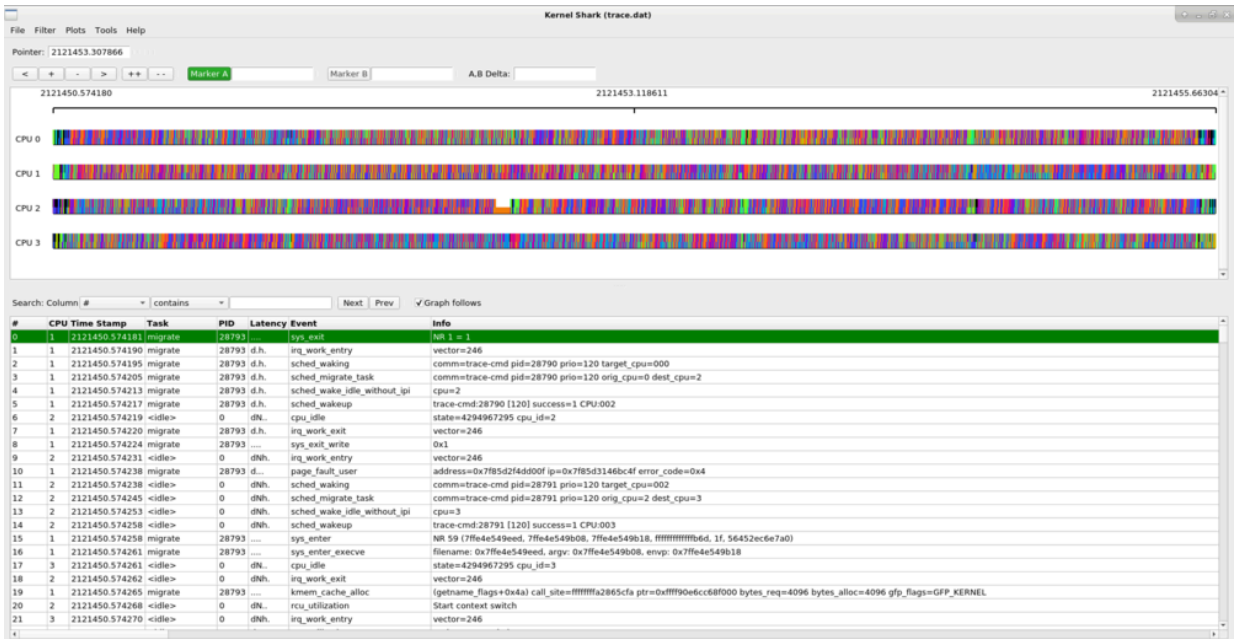
```
% time     seconds  usecs/call     calls      function
------ ----------- ----------- --------- --------------------
 52.64    5.958660     5958660         1 __libc_start_main
 20.64    2.336331     2336331         1 avformat_find_stream_info
 14.87    1.682895         421      3995 strncmp
  7.17    0.811210      811210         1 avformat_open_input
  0.75    0.085290         584       146 av_freep
  0.49    0.055150         434       127 strlen
  0.29    0.033008         660        50 av_log
  0.22    0.025090         464        54 strcmp
  0.20    0.022836       22836         1 avformat_close_input
  0.16    0.017788         635        28 av_dict_free
  0.15    0.016819         646        26 av_dict_get
  0.15    0.016753         440        38 strchr
  0.13    0.014536         581        25 memset
...
------ ----------- ----------- --------- --------------------
100.00   11.318773                  4762 total
```

ftrace

- In-kernel *tracing* functionality
- Can trace
  - Well-defined trace locations in the kernel, called *tracepoints*, identifying important events in the kernel: scheduling, interrupts, etc.
  - Arbitrary functions in the kernel
  - Arbitrary functions in user-space applications
- Low-overhead and optimized tracing
- Accessible using the dedicated *tracefs* filesystem
- `trace-cmd` is a higher-level CLI tool to use *ftrace*
- Can be used to understand overall system activity (what is my system doing?) as well as narrow down specific performance issues
- `https://www.kernel.org/doc/Documentation/trace/ftrace.txt`
- `https://www.trace-cmd.org/`

kernelshark

- Visualization tool for *ftrace* traces
- https://kernelshark.org/



perf

- *instrument CPU performance counters, tracepoints, kprobes, and uprobes*
- Directly included in the Linux kernel source code: tools/perf
- Began as a tool for using the performance counters in Linux, and has had various enhancements to add tracing capabilities
- Supports a list of measurable events: hardware events (cycle count, L1 cache hits/miss, page faults), software events (tracepoints)
- https://perf.wiki.kernel.org

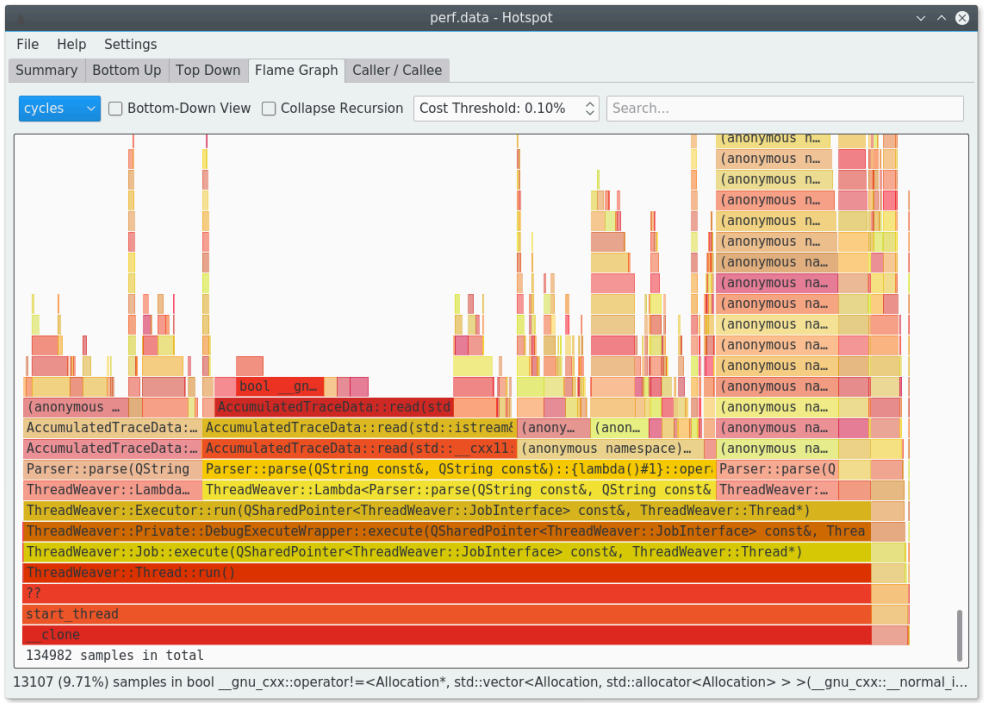
perf examples

- List all currently known events `perf list`
- List scheduler tracepoints ‘perf list `sched:*'`
- CPU counter statistics for the specified command `perf stat <command>`
- CPU counter statistics for the entire system, for 5 seconds `perf stat -a sleep 5`
- Profiling: sample on-CPU functions for the specified command, at 99 Hertz `perf record -F 99 <command>`
- Tracing: trace all context-switches via sched tracepoint, until Ctrl-C `perf record -e sched:sched_switch -a`
- Many more at `https://www.brendangregg.com/perf.html`

perf GUI: hotspot

- Hotspot - the Linux perf GUI for performance analysis
- The main feature of hotspot is visualizing a `perf.data` file graphically
- `github.com/KDAB/hotspot`

perf GUI: hotspot



gprof

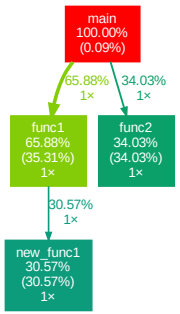
- Application-level profiler
- Part of *binutils*
- Requires passing gcc `-pg` option at build/link time
- Run your program normally, it automatically generates a `gmon.out` file when exiting
- Use the `gprof` tool on `gmon.out` to extract profiling data
- `http://sourceware.org/binutils/docs/gprof/`

gprof example

```
\$\_./test-gprof
\$\_gprof test-gprof gmon.out Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls   s/call   s/call
name
 35.31      7.46     7.46        1     7.46    13.92
func1
 34.03     14.65     7.19        1     7.19     7.19
func2
 30.57     21.11     6.46        1     6.46     6.46
new_func1
  0.09     21.13     0.02
main
[...]
```

gprof example



Generated with `gprof2dot`

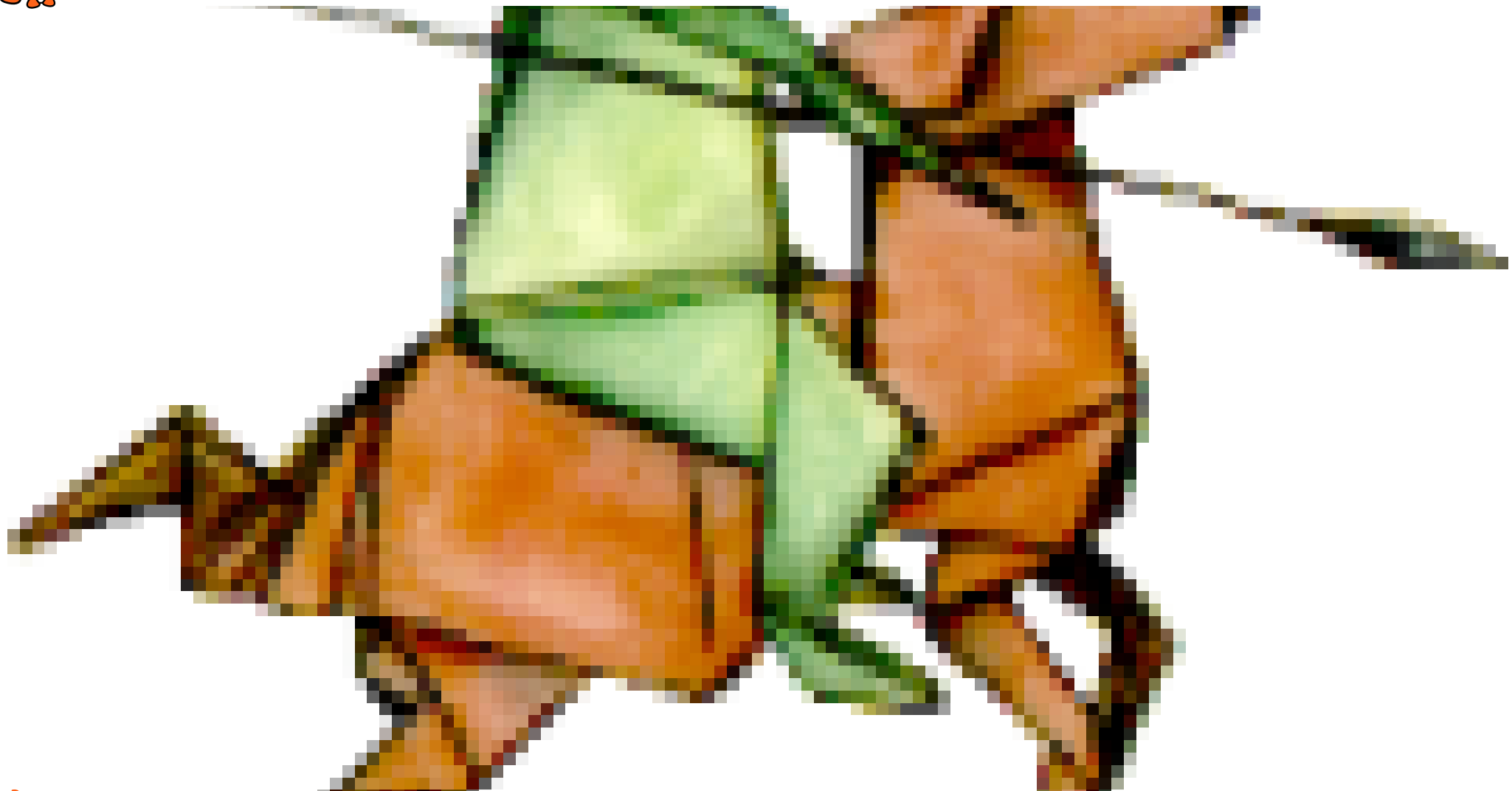
Memory debugging

Valgrind

https://valgrind.org/

- *instrumentation framework for building dynamic analysis tools*
  - detect many memory management and threading bugs
  - profile programs
- Supported architectures: x86, x86-64, ARMv7, ARMv8, mips32, s390, ppc32 and ppc64
- Very popular tool especially for debugging memory issues
- Runs your program on a synthetic CPU $arrow.r$_significant performance impact (100 x slower on SAMA5D3!), but very detailed instrumentation
- Runs on the target. Easy to build with Yocto Project or Buildroot.

Valgrind



Valgrind tools

- *Memcheck*: detects memory-management problems
- *Cachegrind*: cache profiler, detailed simulation of the I1, D1 and L2 caches in your CPU and so can accurately pinpoint the sources of cache misses in your code
- *Callgrind*: extension to Cachegrind, provides extra information about call graphs
- *Massif*: performs detailed heap profiling by taking regular snapshots of a program's heap
- *Helgrind*: thread debugger which finds data races in multithreaded programs. Looks for memory locations accessed by multiple threads without locking.
- More at `https://valgrind.org/info/tools.html`

Valgrind examples

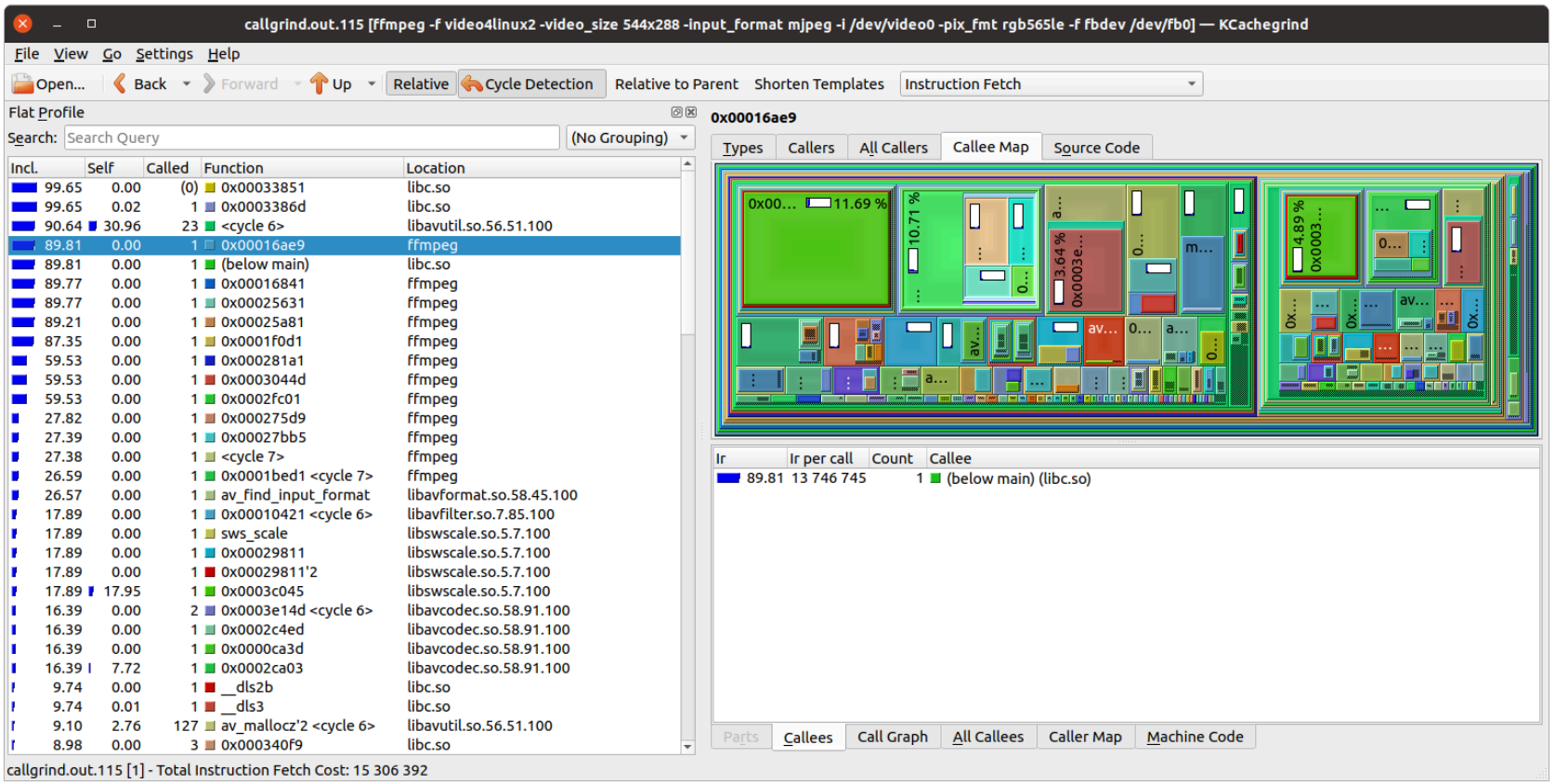
- *Memcheck*

```
\$\_valgrind --leak-check=yes <program>
  ==19182== Invalid write of size 4
  ==19182==    at 0x804838F: f (example.c:6)
  ==19182==    by 0x80483AB: main (example.c:11)
  ==19182==  Address 0x1BA45050 is 0 bytes after a block of size 40 alloc'd
  ==19182==    at 0x1B8FF5CD: malloc (vg_replace_malloc.c:130)
  ==19182==    by 0x8048385: f (example.c:5)
  ==19182==    by 0x80483AB: main (example.c:11)
```

- *Callgrind*

```
\$\_valgrind --tool=callgrind --dump-instr=yes --simulate-cache=yes --collect-jumps=yes <program>
\$\_ls callgrind.out.*
callgrind.out.1234
\$\_callgrind_annotate callgrind.out.1234
```

Kcachegrind - Visualizing Valgrind profiling data

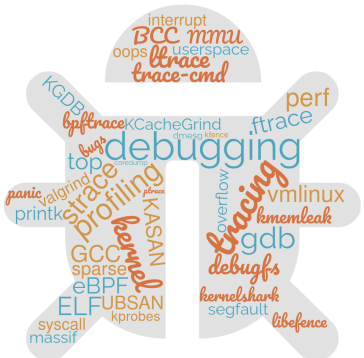


https://github.com/KDE/kcachegrind

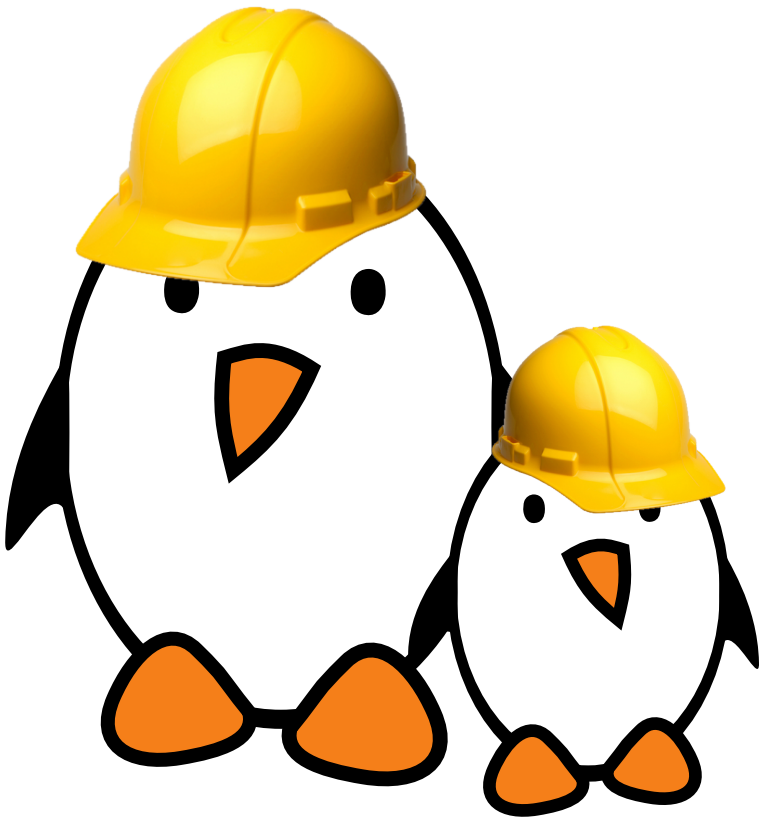
Debugging resources

- Brendan Gregg `Systems performance` book
- Brendan Gregg `Linux Performance` page
- Bootlin's "Linux debugging, profiling, tracing and performance analysis" training course and free training materials (250 pages): `https://bootlin.com/training/debugging/`.

Debugging resources



Practical lab - Application development and debugging



)

- Creating an application that uses an I2C-connected joystick to control an audio player.
- Setting up an IDE to develop and remotely debug an application.
- Using *strace*, *ltrace*, *gdbserver* and *perf* to debug/investigate buggy applications on the embedded board.